

Perceptrons

1 Introduction

Before today's "deep" networks, the **Perceptron** was one of the first algorithms to help computers learn from data.

It's the **building block** of modern neural networks.

Think of it as the simplest model of how a brain cell (neuron) might make decisions.

2 What is a Perceptron?

- Invented by **Frank Rosenblatt (1957)**.
- Purpose: A machine that can learn to classify things into two categories (yes/no, 1/0, true/false).

How it works (step-by-step)

1. **Inputs:** Takes several binary or numerical inputs (**e.g., x_1, x_2, x_3** ..).
2. **Weights:** Each input has a weight (**w_1, w_2, w_3**) tells how important that input is.
3. **Weighted sum:**

$$Z = w_1 \times 1 + w_2 \times 2 + \dots + b$$

(where b is the bias — a constant to shift the decision boundary).

4. **Activation:** Apply a simple rule (often a **step function**) — if $z > 0$, output 1; else 0.

👉 The perceptron "fires" only if the sum is big enough.

3 Perceptron vs Neuron

Aspect	Perceptron	Neuron (modern)
Origin	Proposed by Rosenblatt, 1957.	General concept (biological or artificial).
Activation	Step function (on/off).	Many options: sigmoid, tanh, ReLU, etc.
Output	Binary (0 or 1).	Can produce a range of values.
Use	Basic linear classifier.	Building block of deep networks.

💡 Takeaway:

A perceptron is like a *primitive neuron*. Modern neurons generalize it with smoother activations and can be stacked to form deep networks.

4 Geometric Intuition

- A perceptron is basically a **line (or plane)** separating data points.
- In 2D: it draws a straight line between two classes.
- In 3D: it's a flat plane.

- In higher dimensions: it's a "hyperplane."

If the data is linearly separable (you can split it with a single line), the perceptron can find that line.

But if the data isn't separable with a straight line (e.g., the XOR problem), a single perceptron **can't** solve it — that's why we need multi-layer networks.

5 Code Example

A simplified Python-like pseudocode:

```
# inputs, weights, and bias
x = [x1, x2, x3]
w = [w1, w2, w3]
b = bias

# weighted sum
z = sum(x[i] * w[i] for i in range(len(x))) + b

# activation (step)
output = 1 if z > 0 else 0
```

Training adjusts the weights w and bias b to get the right output.

6 Outro

- The perceptron was the **first stepping stone** toward deep learning.
- It taught us how to model a decision mathematically.
- While limited (can't handle complex patterns alone), its ideas live on in every modern neuron.

Quick Recap

- **Perceptron:** Early algorithm → weighted sum + step activation → binary output.
 - **Neuron:** More flexible → smooth activations → can be combined into layers.
 - **Geometry:** Think of a perceptron as drawing a line (or plane) to divide two groups.
-